

Training-free Context-adaptive Attention for Efficient Long Context Modeling

Zeng You, Yaofu Chen, Shuhai Zhang, Zhijie Qiu, Tingyu Wu, Yingjian Li, Yaowei Wang, Mingkui Tan

Abstract—Large Language Models (LLMs) have demonstrated remarkable capabilities across a wide range of natural language processing tasks. These capabilities stem primarily from the self-attention mechanism, which enables the modeling of long-range dependencies. However, the quadratic complexity of self-attention with respect to sequence length poses significant computational and memory challenges, especially as sequence length extends to extremes. While various sparse attention and KV cache compression methods have been proposed to improve efficiency, they often suffer from limitations such as reliance on fixed patterns, inability to accelerate both prefilling and decoding stages, or the requirement for additional training. In this paper, we propose Training-free Context-adaptive Attention (TCA-Attention), a training-free sparse attention mechanism that selectively attends to only the informative tokens for efficient long-context inference. Our method consists of two lightweight phases: i) an offline calibration phase that determines head-specific sparsity budgets via a single forward pass, and ii) an online token selection phase that adaptively retains core context tokens using a lightweight redundancy metric. TCA-Attention provides a unified solution that accelerates both prefilling and decoding while reducing the KV cache memory footprint, without requiring parameter updates or architectural changes. Theoretical analysis shows that our approach maintains bounded approximation error. Extensive experiments demonstrate that TCA-Attention achieves a $2.8\times$ speedup and reduces KV cache by 61% at 128K context length while maintaining performance comparable to full attention across various benchmarks, offering a practical plug-and-play solution for efficient long-context inference.

Index Terms—Sparse Attention, Large Language Models, Long Context Modeling.

I. INTRODUCTION

LARGE language models (LLMs) have revolutionized numerous natural language processing tasks, establishing

themselves as indispensable tools for modern AI applications [1]–[4]. Models such as the LLaMA series [5], [6], GPT-family models [7], [8], and recent breakthroughs like GPT-o1 [8] and DeepSeek-R1 [9] have demonstrated exceptional capabilities in complex tasks, including multi-step reasoning [10]–[12] and document-level comprehension [13], [14]. The remarkable performance of these models predominantly stems from the self-attention mechanism [15], which enables comprehensive dependency modeling across entire input sequences through dense pairwise token interactions.

However, this expressive power comes at a substantial computational cost, posing a significant challenge for efficient long context modeling. As context lengths extend to extremes (e.g., 128K tokens), the quadratic complexity of self-attention presents severe bottlenecks in both computation and memory. For a sequence of length L , the prefilling stage, which computes attention across all token pairs, exhibits $\mathcal{O}(L^2)$ computational complexity, while the decoding stage suffers from $\mathcal{O}(L)$ memory access overhead per generated token due to the linearly growing key-value (KV) cache. More critically, not all tokens contribute equally to the final output. The presence of redundant or minimally informative tokens dilutes attention allocation to semantically critical content, thereby degrading both efficiency and accuracy [16], [17]. This challenge becomes increasingly pronounced in applications involving lengthy documents, extended conversations, or complex multimodal contexts.

To mitigate these issues, existing approaches have pursued several distinct directions. Early efforts, such as static sparse attention [18]–[20], employ predefined attention patterns to reduce computation, yet they lack adaptability to input content and often compromise performance on context-sensitive tasks. More recent prefilling-stage dynamic methods [21]–[23] adjust attention patterns per head during prefilling, but they are restricted to a limited set of handcrafted sparsity patterns and do not address the inefficiency in the decoding phase. On the other hand, decoding-stage methods mainly follow two directions. KV cache compression techniques [24], [25] reduce memory usage and computation by evicting or merging cached KV entries, but cannot accelerate prefilling and often apply uniform compression strategies across attention heads. Another line of work, represented by Quest [26] and subsequent KV-retrieval methods [27], [28], retains the complete KV cache but retrieves only a query-relevant subset for attention computation at each decoding step. Although these methods reduce decoding-time KV access, they neither accelerate pre-

Zeng You is with the School of Future Technology, South China University of Technology, Guangzhou, China and also with Pengcheng Laboratory, Shenzhen, China (e-mail: zengyou.yz@gmail.com).

Yaofu Chen is with the School of Future Technology, South China University of Technology, Guangzhou, China (e-mail: chenyaofu@gmail.com).

Shuhai Zhang, Zhijie Qiu, and Tingyu Wu are with the School of Software Engineering, South China University of Technology, Guangzhou, China (e-mail: shuhai Zhangshz@gmail.com, 202420145073@mail.scut.edu.cn, 202230490150@mail.scut.edu.cn).

Yingjian Li is with Pengcheng Laboratory, Shenzhen, China (e-mail: yingjian1122@gmail.com).

Yaowei Wang is with Harbin Institute of Technology, Shenzhen, China, and also with Pengcheng Laboratory, Shenzhen, China (e-mail: wangyaowei@hit.edu.cn).

Mingkui Tan is with the School of Software Engineering, South China University of Technology, Guangzhou 510006, China (e-mail: mingkui-tan@scut.edu.cn).

Zeng You, Yaofu Chen, and Shuhai Zhang are equal contributors.

Mingkui Tan and Yaowei Wang are corresponding authors.

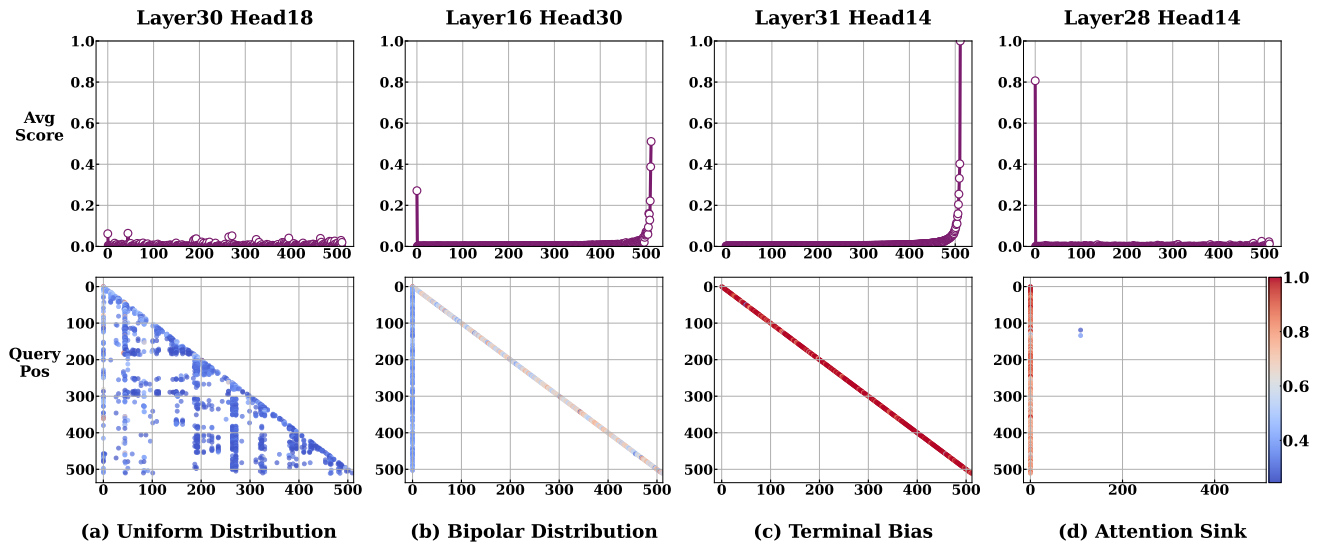


Fig. 1: Visualizations of attention distributions in LLaMA3.1-8B-Instruct: 1) average attention scores across key positions on the first row, and 2) scatter points highlighting attention scores above row-wise averages on the second row. The observations motivate the design principles of TCA-Attention (refer to Appendix B for more visualizations).

TABLE I: Characteristic comparisons between existing methods and our TCA-Attention.

Method	Training-free	Adaptive Selection	Prefilling Accel.	Decoding Accel.	KV Cache Reduction
MInference [21]	✓	✓	✓		
FlexPrefill [22]	✓	✓	✓		
XAttention [23]	✓	✓	✓		
SnapKV [24]	✓	✓		✓	✓
CAKE [25]	✓	✓		✓	✓
Quest [26]	✓	✓		✓	
DuoAttention [29]			✓	✓	✓
TCA-Attention (Ours)	✓	✓	✓	✓	✓

filling nor directly reduce the total KV cache storage. While a few unified frameworks, such as DuoAttention [29] and Lserve [30], target both prefilling and decoding, they require additional training to determine head-specific patterns and only enable context-adaptive sparsity during decoding.

The fundamental limitations of existing literature can be distilled into two critical research gaps. First, despite growing interest in efficient attention mechanisms, most existing methods fail to provide a full *training-free* solution that dynamically accelerates both prefilling and decoding while simultaneously reducing KV cache footprint, without requiring architectural modifications or parameter updates. Second, they fail to model the intricate nature of attention redundancy adequately. As demonstrated in Figure 1 different attention heads exhibit diverse redundancy distributions across models, ranging from uniform distributions to attention sinks, aligning with findings in prior works [19]–[21]. Meanwhile, token importance varies dynamically within input content. However, existing methods either use uniform compression strategies or rely on a limited set of hand-designed patterns, lacking the fine-grained, context-adaptive mechanism required to handle such head-specific and context-dependent redundancy.

To address these limitations, we propose Training-free Context-adaptive Attention (TCA-Attention), a sparse attention mechanism designed for efficient long context modeling across prefilling and decoding stages. In contrast to prior

unified approaches [20], [29], [30], our method inherently requires no parameter updates, architectural adjustments, or reliance on predefined sparse patterns. It operates via two complementary phases: 1) an *offline sparsity configuration phase* that estimates head-specific redundancy levels through a single forward pass, determining individualized sparsity budgets per head; 2) an *online core-context selection phase* that dynamically selects a compact set of core tokens per head using a context-aware redundancy metric during inference.

This design enables TCA-Attention to be simultaneously head-aware, context-adaptive, and training-free. As summarized in Table I, unlike prefilling-only sparse attention, decoding-only KV compression or retrieval, and training-dependent unified approaches, TCA-Attention accelerates both inference stages while directly reducing the resident KV cache footprint. Moreover, we provide a theoretical guarantee showing that the approximation error introduced by the selected core context is explicitly bounded and controllable. Our contributions are summarized as follows:

- Our proposed TCA-Attention unifies acceleration for prefilling, decoding, and KV cache reduction via dynamically attending to only the informative tokens in the context. Unlike prior approaches relying on predefined patterns or retraining, our method requires no additional training and dynamically adapts sparsity to input content, enabling plug-and-play deployment for efficient long-context inference.

- Our proposed TCA-Attention integrates two phases that explicitly adapt to both head-specific and context-dependent attention redundancy: 1) an offline calibration phase for head-specific sparsity configuration, and 2) an online token selection phase with a lightweight, context-aware redundancy metric. This enables fine-grained acceleration while preserving long context modeling performance.
- We prove that the approximation error introduced by core context selection is explicitly bounded, ensuring reliability without training. This principled design is validated empirically. Extensive experiments demonstrate that our method achieves a $2.8\times$ speedup and reduces KV cache by **61%** at 128K context length while maintaining performance comparable to full attention across various tasks.

II. RELATED WORK

Efficient Attention. Efficient attention mechanisms address the quadratic complexity of self-attention and KV cache memory bottlenecks through several major strategies. Early static sparse attention mechanisms [18]–[20] reduce computational complexity by imposing predefined sparsity patterns on attention matrices. While effective in certain long-document tasks and easy to deploy, these approaches lack adaptivity. Another line of work [31], [32] reformulates self-attention as a linear dot-product of kernel feature maps and leverages the associativity property of matrix products to achieve linear complexity. These methods reduce theoretical complexity and memory usage, but often introduce accuracy degradation. A complementary direction improves the efficiency of exact attention through IO-aware kernel optimizations, tiling strategies, and parallel execution primitives [33]–[36]. Such implementations significantly boost runtime performance and memory efficiency for full attention. Our method is orthogonal to this direction.

More recent work [21]–[23], [37] proposes dynamic or training-free sparse attention mechanisms that target the pre-filling stage, where the entire input sequence is processed at once. These methods typically select patterns from predefined sets at the head level. While they achieve acceleration during prefilling, they do not reduce KV cache growth and thus provide limited benefits for autoregressive decoding. During generation, the primary bottleneck is the linear growth of the KV cache. Decoding-stage methods mainly follow two directions. KV cache compression techniques reduce computation and memory usage during generation by evicting [24], [25] or merging [38] cached tokens. However, they cannot accelerate prefilling and often apply uniform compression policies across attention heads. In contrast, KV-retrieval methods [26]–[28], represented by Quest [26], retain the complete KV cache but retrieve only a query-relevant subset for attention computation at each decoding step. Although these approaches reduce decoding-time KV access and attention computation, they do not directly reduce the total KV states that must be retained and provide no acceleration for prefilling. Recent studies [16], [17], [39], [40] like DuoAttention [29] and Lserve [30] attempt to bridge both stages using consistent sparsity patterns. However, they often require additional model training, complex profiling, or intrusive system-level changes.

In contrast to the above categories, our TCA-Attention provides a training-free, head-aware, and context-adaptive sparse attention mechanism that simultaneously accelerates prefilling and decoding. It reduces computation and KV cache size within a single unified framework, while offering explicit theoretical error guarantees and requiring no model retraining, architecture modification, or heavy offline tuning.

Long-Context Language Modeling. Handling long input sequences is critical for applications requiring document-level reasoning, code generation, or extended dialogue history [41]–[43]. From a data perspective, tailored curation and synthesis strategies are developed to equip models with long-range dependency awareness. For the pre-training stage, the methods focus on filtering for long-context quality via linguistic metrics or attention-pattern analysis, mixing domains to preserve diversity while upsampling long sequences, and synthesizing data by clustering related passages or constructing “knotted” documents to enhance structural reasoning [44]–[46]. During the post-training stage, instruction data are synthesized to mitigate positional bias and to support multi-hop reasoning over dispersed information [41], [47].

Architecturally, Rotary Position Embedding (RoPE) [48] and its variants serve as the foundation; extrapolation techniques such as NTK-aware scaling [49], and YaRN [50] systematically map out-of-distribution positions into the training range, often without fine-tuning. Beyond embeddings, a significant direction replaces quadratic attention with linear-complexity models, such as selective state-space models (Mamba) [51], the RetNet architecture with recurrent and parallel paradigms [32], and Test-Time Training that dynamically stores context in fast weights [52]. These efforts enable models to scale to longer contexts. Our work aligns with this goal by introducing a training-free, adaptive sparse attention mechanism that accelerates both prefilling and decoding.

III. MOTIVATIONS

A. Understanding Bottlenecks of Self-Attention in Long-context Modeling

Most existing LLMs are built on the Transformer [15] architecture, where the self-attention mechanism serves as the core module for capturing global contextual dependencies. Given an input sequence $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_L\} \in \mathbb{R}^{L \times d}$ of L tokens with model dimension d , the multi-head self-attention mechanism computes contextualized representations through h heads:

$$\begin{aligned} \text{MultiHead}(\mathbf{X}) &= [\text{Att}^1, \dots, \text{Att}^h] \in \mathbb{R}^{L \times d}, \\ \text{Att}^i &= \text{softmax} \left(\frac{\mathbf{Q}^i \mathbf{K}^{i\top}}{\sqrt{d_h}} \right) \mathbf{V}_i, \\ \mathbf{Q}^i &= \mathbf{X} \mathbf{W}^{Q_i}, \mathbf{K}^i = \mathbf{X} \mathbf{W}^{K_i}, \mathbf{V}_i = \mathbf{X} \mathbf{W}^{V_i}, \end{aligned} \quad (1)$$

where d_h is the head dimension (typically $d_h = d/h$), $\mathbf{W}^{Q_i}, \mathbf{W}^{K_i}, \mathbf{W}^{V_i} \in \mathbb{R}^{d \times d_h}$ are learnable parameters for the i -th head. The multi-head mechanism enables parallel attention operations across distinct feature subspaces, facilitating position-aware information aggregation.

Challenges of Self-Attention in Handling Context Redundancy: As the context length L grows, the context

inevitably exhibits redundant information [16], [17], [21]. Vanilla self-attention faces three challenges in handling such redundancy: 1) **Quadratic computational complexity**: It incurs $O(L^2)$ computational cost by computing pairwise attention scores across all tokens. This leads to excessive computation when much of the context is redundant. 2) **Memory growth in key-value (KV) Cache**: The KV cache, which grows linearly with L , presents a major deployment bottleneck. For instance, processing a 128K sequence with LLaMA2-7B requires 64GB GPU memory for KV cache, exceeding the capacity of most GPUs. 3) **Interference from irrelevant tokens**: More critically, irrelevant tokens degrade the model's ability to focus on critical information, thereby harming performance.

B. Exploration and Exploitation of Attention Redundancy Properties

The severe inefficiency of self-attention in long contexts raises a critical question: *given the aforementioned head-specific and context-dependent nature of attention redundancy, how can we design a unified framework that exploits these properties to accelerate both prefilling and decoding?* As visualized in Figure 1 and supported by prior works [20]–[23], attention redundancy manifests heterogeneously across heads and varies dynamically with input context. The fundamental challenge, however, lies not in observing these properties but in addressing them simultaneously and effectively within a single, training-free acceleration framework.

Existing approaches only address a subset of this challenge. *Static sparse methods* [18]–[20] ignore both dynamic context and head specificity. *Prefilling-only dynamic attention methods* [21], [22] adapt per head but are constrained to a small set of hand-designed patterns and, critically, fail to accelerate decoding. *Decoding-only KV compression methods* [24], [25], [38] reduce memory but cannot accelerate prefilling and often apply uniform policies across heads within a layer. While recent unified methods like DuoAttention [29] and Lserve [30] target both stages, they require continuous training for pattern determination and operate with fixed sparse patterns, lacking context-aware adaptability.

Design Principles for Efficient Attention. This limitation reveals a fundamental design gap: no existing method simultaneously achieves dynamic context adaptation, head-aware sparsity, and training-free deployment across both prefilling and decoding stages. We therefore distill the requirements for an ideal solution into four key principles:

- 1) **Sparse Computation**: Selectively attend to a small subset of critical tokens and discard irrelevant ones due to the tokens' redundancy in both prefilling and decoding stages.
- 2) **Dynamic Adaptation**: The sparsity must be input-dependent, not predetermined by fixed sparse patterns.
- 3) **Head-aware Sparsity Customization**: Sparsity strategies must be tailored to the redundancy level of each head.
- 4) **Training-free Deployment**: Eliminates the need for parameter updates, architectural modifications, or retraining, enabling immediate plug-and-play deployment without compromising model performance.

Existing methods fulfill only a subset of these principles, creating a performance-efficiency gap. Static methods violate (2) and (3); prefilling-only and decoding-only methods violate (1); existing unified approaches violate (2) and (4). This motivates our design of a unified attention mechanism that satisfies all four principles simultaneously, *i.e.*, sparsity, dynamic, head-aware customization, and training-free.

IV. TRAINING-FREE CONTEXT-ADAPTIVE ATTENTION

In this paper, we propose Training-free Context-adaptive Attention (TCA-Attention) that simultaneously accelerates prefilling, reduces decoding latency, and compresses KV cache by dynamically selecting core tokens in a head-specific and context-aware manner. Our method is completely training-free, which ensures seamless integration into existing LLMs.

Overview of Training-free Context-adaptive Attention.

Our TCA-Attention includes two phases: 1) During the offline sparsity configuration determination phase (Section IV-A), we adopt a small calibration dataset to estimate the redundancy level of each head and determine its appropriate sparsity configuration (*i.e.*, the number of tokens to preserve). This phase is performed only once per model. 2) During inference, we dynamically select active tokens in each head based on the determined sparsity configuration (Section IV-B). Theoretically, we prove that the approximation error of TCA-Attention is bounded and controllable (Section IV-C). We implement our method using Triton to enable efficient parallelization across attention heads. We illustrate the overview in Figure 2 and present the pseudocode in Algorithms 1 and 2.

To capture both long-range and short-range dependencies, our dynamic token selection operates on two complementary components: *a global subset* and *a local subset*. We dynamically select the global subset $\mathbf{K}^G = \mathbf{K}_S$ and $\mathbf{V}^G = \mathbf{V}_S$ from the entire context based on the head-specific sparsity configuration, where $S \subseteq \{1, \dots, L\}$ is the selected token index set. We detail the computation of S in Section IV-B. This subset is responsible for modeling long-distance dependencies. On the other hand, we always preserve the most recent w tokens for each query as the local subset $\mathbf{K}^L$ and $\mathbf{V}^L$ to capture fine-grained local context, a critical element highlighted by prior works [20], [53], [54] and our analysis. Notably, we ensure no overlap between the global and local subsets to avoid duplicated computation. Given the query matrix $\mathbf{Q} \in \mathbb{R}^{L \times d_h}$, our TCA-Attention computes the attention output as follows:

$$\text{Att} = \text{Softmax} \left(\frac{\mathbf{Q}[\mathbf{K}^G; \mathbf{K}^L]^\top}{\sqrt{d_h}} \right) [\mathbf{V}^G; \mathbf{V}^L]. \quad (2)$$

Since the resulting output $\text{Att} \in \mathbb{R}^{L \times d_h}$ preserves the original sequence length L , our method seamlessly integrates into existing LLM architectures without requiring any structural modifications. For clarity and brevity, we describe all operations with respect to a single attention head in the following. We apply the same mechanism independently and in parallel to all heads in all layers.

A. Offline Head-Specific Sparsity Determination

Motivated by the head-specific redundancy property observed in Figure 1 and prior works [18], [20], [21], we

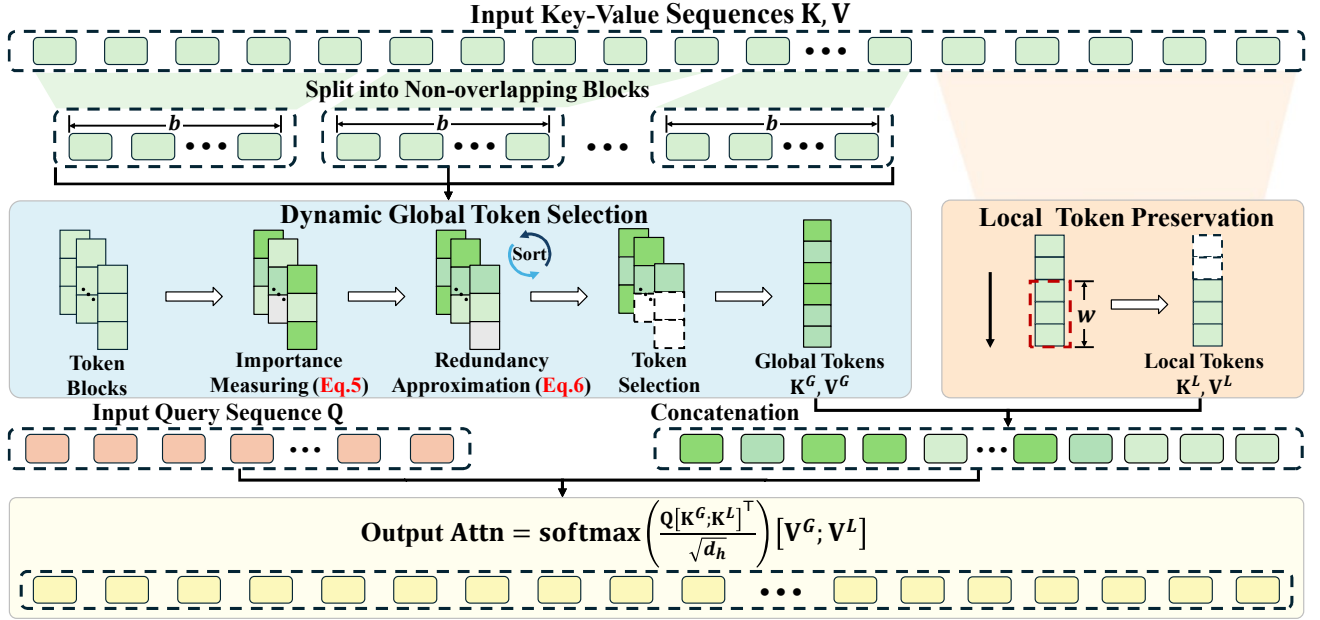


Fig. 2: Overview of our TCA-Attention. We dynamically select a subset of key/value tokens, which combines 1) a global subset $\mathbf{K}^G, \mathbf{V}^G$, selected online based on a predetermined configuration (Section IV-B), to model long-distance dependencies; and 2) a local subset $\mathbf{K}^L, \mathbf{V}^L$, preserving neighboring tokens to capture fine-grained local context. The concatenation of these complementary subsets is used for the final attention computation in Eq. (2).

Algorithm 1 Offline Head-Specific Sparsity Determination

Require: Queries $\mathbf{Q} \in \mathbb{R}^{L \times d_h}$, keys $\mathbf{K} \in \mathbb{R}^{L \times d_h}$, configuration candidates $\mathcal{C} = \{\mathbf{p}^1, \dots, \mathbf{p}^M\}$, threshold $\tau \in [0, 1]$.

- 1: Compute attention $\mathbf{A} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d_h})$
- 2: Initialize valid set $\mathcal{C}_{\text{valid}} \leftarrow \emptyset$
- 3: **for** each configuration $\mathbf{p}^i \in \mathcal{C}$ **do**
- 4: Compute the selected token indices $\mathcal{S}_i$ based on $\mathbf{p}^i$ as described in Section IV-B
- 5: Compute aggregated score $\mathbf{a}_i$ via Eq. (4)
- 6: **if** $\mathbf{a}_i \geq \tau$ **then** $\mathcal{C}_{\text{valid}} \leftarrow \mathcal{C}_{\text{valid}} \cup \{\mathbf{p}^i\}$
- 7: **end for**
- 8: $\mathbf{p}^* = \text{argmin}_{\mathbf{p}^i \in \mathcal{C}_{\text{valid}}} |\mathcal{S}_i|$

Ensure: Configuration $\mathbf{p}^*$

Algorithm 2 Inference with Online Core Context Selection

Require: Queries $\mathbf{Q} \in \mathbb{R}^{L \times d_h}$, keys $\mathbf{K} \in \mathbb{R}^{L \times d_h}$, values $\mathbf{V} \in \mathbb{R}^{L \times d_v}$, block size b , window size w , configuration $\mathbf{p}^*$ from Alg. 1

- 1: Compute token scores $\mathbf{s} = \text{softmax}(\mathbf{Q}_L, \mathbf{K}^\top / \sqrt{d_h})$
- 2: Split $\mathbf{K}$ into $m = \lfloor L/b \rfloor$ blocks (same for $\mathbf{V}$)
- 3: Compute block redundancy score $\mathbf{h} = [h_1, \dots, h_m]$ via Eq. (6)
- 4: Sort blocks in ascending order by the redundancy score $\mathbf{h}$
- 5: Compute token requirements $\mathbf{t} = [t_1, t_2, \dots, t_m]$ based on configuration $\mathbf{p}^*$ via Eq. (7)
- 6: Generate the selected token indices $\mathcal{S}$ via Eq. (8)
- 7: $\mathbf{Att} = \text{Softmax}(\mathbf{Q}[\mathbf{K}^G; \mathbf{K}^L]^\top / \sqrt{d_h}) [\mathbf{V}^G; \mathbf{V}^L]$,
 $\mathbf{K}^G = \mathbf{K}_{\mathcal{S}}, \mathbf{V}^G = \mathbf{V}_{\mathcal{S}}, \mathbf{K}^L = \mathbf{K}_{L-w+1:, \mathcal{S}}, \mathbf{V}^L = \mathbf{V}_{L-w+1:, \mathcal{S}}$

Ensure: Attention output $\mathbf{Att}$

introduce a one-time offline calibration to determine a unique sparsity configuration for each attention head. This configuration dictates how many tokens (token budget) the head should preserve during inference based on its inherent redundancy level. To handle a sequence $\mathbf{X}$ with arbitrary length L , we operate on a block-wise basis. We partition the input sequence $\mathbf{X}$ into $m = \lfloor L/b \rfloor$ non-overlapping blocks of size b . Let $\mathcal{K} = \{1, 2, 4, \dots, b\}$ denote the discrete set of allowable per-block retain counts. For each head, we seek a configuration $\mathbf{p} = [p_k | k \in \mathcal{K}]$, where $\sum_{k \in \mathcal{K}} p_k = 1$. Each $p_k \in \mathbf{p}$ represents the proportion of blocks assigned a budget of k tokens, enabling an adaptive policy: the actual number of tokens preserved per block can vary based on its content, while the overall distribution is governed by $\mathbf{p}$.

Gaussian-Sampling Configuration Candidates Generation. Given the diverse redundancy levels across heads, we need a set of candidate configurations that smoothly transition from high-efficiency to high-accuracy modes. Creating such a

diverse configuration set manually is impractical, as it would require tuning dozens of hyperparameters across all heads. To solve this, we propose a *log-Gaussian sampling strategy* that generates candidate configurations $\mathcal{C} = \{\mathbf{p}^1, \mathbf{p}^2, \dots, \mathbf{p}^M\}$ from efficiency-focused to accuracy-focused, controlled by just two intuitive hyperparameters. Specifically, we model the probability of retaining k tokens using a log-Gaussian distribution centered around μ :

$$p_k = \frac{\Phi[\log_2(k)]}{\sum_{k' \in \mathcal{K}} \Phi[\log_2(k')]} \text{, where } \Phi[x] = \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right), \quad (3)$$

where μ controls the center of the token budget in log-space (a larger μ prioritizes performance via more token requirement) and σ regulates diversity in sampled configurations (a larger σ explores extreme token budgets). In practice, we generate M configuration candidates by uniformly sweeping μ from 0 to $\log_2(b)$ with a fixed σ : $\mathcal{C} = \{\mathbf{p}^1, \mathbf{p}^2, \dots, \mathbf{p}^M\}$, where the i -th configuration

$\mathbf{p}^i$ is computed with $\mu^i = \log_2 \left(1 + \frac{(i-1) \cdot (b-1)}{M-1} \right)$. This ensures smooth interpolation between efficiency-oriented and accuracy-oriented sparsity patterns.

Cumulative-Score Configuration Determination. The goal of offline determination is to find the most efficient (sparsest) configuration from the candidate set $\mathcal{C}$ that maintains the head's performance. We measure performance via the aggregated attention score retained by the selected tokens. For a given candidate $\mathbf{p}^i$, we simulate its token selection on a calibration dataset (using our online mechanism from Section IV-B) to obtain the set of indices $\mathcal{S}_i$. We then compute the sum of the average attention scores for each selected token:

$$\mathbf{a}_i = \sum_{k \in \mathcal{S}_i} \frac{1}{\|\mathbf{A}_{:,k}\|_0} \sum_{j=1}^{\|\mathbf{A}_{:,k}\|_0} \mathbf{A}_{j,k}, \quad (4)$$

where $\mathbf{A} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d_h}) \in \mathbb{R}^{L \times L}$ is computed on a calibration dataset, $\|\mathbf{A}_{:,k}\|_0$ denotes the number of non-zero elements in the k -th column of $\mathbf{A}$. We select the configuration $\mathbf{p}^* = \arg\min_{\mathbf{p}^i \in \mathcal{C}_{\text{valid}}} |\mathcal{S}_i|$, where $\mathcal{C}_{\text{valid}}$ denotes a configuration set with each $\mathbf{p}^i$ satisfying $\mathbf{a}_i \geq \tau$ and $|\mathcal{S}_i|$ denotes the number of preserved tokens. This selects the configuration that preserves the fewest tokens while retaining at least a fraction τ of the aggregated attention mass. This is a one-time, model-specific process, resulting in a task-agnostic configuration that generalizes across inputs.

B. Online Core Context Selection

During inference, the offline-determined configuration $\mathbf{p}^*$ for each head dictates its overall token budget. The goal of the online stage is to distribute this budget adaptively across the input sequence based on the current context, thereby reducing the context-dependent redundancy. For an input sequence of length L , we first obtain a global, token-level importance score $\mathbf{s} \in \mathbb{R}^L$. We compute $\mathbf{s}$ using the query vector of the last token, which in causal self-attention has full visibility over the entire sequence and thus serves as a natural context summarizer. This allows us to identify tokens that are most relevant to the current generation. We compute the importance score as follows:

$$\mathbf{s} = \text{softmax} \left(\frac{\mathbf{Q}_{L,:} \mathbf{K}^\top}{\sqrt{d_h}} \right), \quad (5)$$

where $\mathbf{Q}_{L,:}$ is the query vector of the last token. Under causal attention, the last query has access to the complete input prefix and directly conditions the subsequent token generation, making it a compact query-aware summary of the current context. Compared with importance estimation based on the last k queries, using only the last query reduces the scoring complexity from $\mathcal{O}(kL)$ to $\mathcal{O}(L)$. We use $k = 1$ in all main experiments and evaluate its accuracy-efficiency trade-off against multi-query variants in Section V-C. Then, similar to the offline phase, we partition the input sequence into non-overlapping blocks of size b . Any trailing tokens that do not form a complete block are assigned to the local subset.

For each block $\mathcal{B}_j$, we compute a redundancy score $\mathbf{h}_j$ that quantifies its information density:

$$\mathbf{h}_j = (1 - \alpha) \cdot \sum_{i \in \mathcal{B}_j} \mathbf{s}_i + \alpha \cdot \left(1 - \frac{\sum_{i \in \mathcal{B}_j} \mathbf{s}_i^2}{\left(\sum_{i \in \mathcal{B}_j} \mathbf{s}_i \right)^2} \right), \quad (6)$$

The first term (weighted by $1 - \alpha$) penalizes blocks with low total attention mass. The second term (weighted by α) is a variant of the Herfindahl-Hirschman Index [55]; it decreases as attention becomes more concentrated on a few tokens (indicating higher redundancy within the block). Thus, a lower $\mathbf{h}_j$ score indicates a more redundant block. The blocks are then sorted by their $\mathbf{h}_j$ scores, resulting in an ordered list of block indices $\mathbf{I} = \text{SortIndex}(\mathbf{h})$.

In the online selection, we adaptively assign a token budget $\mathbf{t}_i$ to each block, dictated by its rank in $\mathbf{I}$ and the head's predefined configuration $\mathbf{p}^*$. Recall that $\mathbf{p}^* = [p_1, p_2, \dots, p_b]$ defines the head's sparsity policy: it specifies that a proportion p_k of blocks should be assigned a budget of k tokens. We enforce this policy through a deterministic mapping:

$$\mathbf{t}_i = \Psi[\mathbf{I}_i], \text{ where } \Psi = \left(\underbrace{1, \dots, 1}_{[m \times p_1]}, \underbrace{2, \dots, 2}_{[m \times p_2]}, \dots, \underbrace{b, \dots, b}_{[m \times p_b]} \right). \quad (7)$$

This mechanism ensures that the most information-dense blocks (highest rank in $\mathbf{I}$) receive the largest token budgets (later entries in Ψ), as dictated by the head's sparsity configuration $\mathbf{p}^*$. For each block $\mathcal{B}_i$, we select the top- $\mathbf{t}_i$ tokens with the highest importance scores $\mathbf{s}$:

$$\mathcal{S}_i = \text{Top}(\mathbf{t}_i; \mathcal{B}_i). \quad (8)$$

The union of all these subsets $\mathcal{S} = \bigcup_{i=1}^m \mathcal{S}_i$ forms the global subset. This set, concatenated with the local subset of the most recent w tokens, is used to compute the final attention output via Eq. (2). As analyzed in Section V-D, the overhead of our lightweight redundancy metric is marginal compared to the significant computational savings achieved by sparsification.

Online Token Selection in Decoding Stage. Our method naturally extends to the decoding stage, where tokens are incrementally generated. Specifically, once the number of new tokens reaches the block size b , we evaluate the importance of its tokens using the same scoring function in Eq. (5) based on the last query. We then retain only the top- t most informative tokens in the block based on their scores. The value $t = \lfloor \sum_{k \in \mathcal{K}} k \cdot p_k \rfloor$ corresponds to the average number of tokens preserved per block for that head, derived from the determined configuration $\mathbf{p}^*$. This ensures that only the relevant context is kept, enabling efficient and effective long-sequence modeling in the decoding stage.

C. Theoretical Guarantees for TCA-Attention

A desirable property of any sparse attention mechanism is the ability to approximate full attention with bounded and predictable deviation. To characterize this behavior, we provide a theoretical analysis showing that the approximation error introduced by TCA-Attention is explicitly controlled by

TABLE II: Comparisons on LongBench-E [42] across efficient-attention methods under LLaMA3.1-8B and Qwen2.5-7B models. We report the attention computation latency at 64K context.

Method	Sin. Doc. QA	Mul. Doc. QA	Sum.	Few Shot	Syn.	Code	Average	Latency (ms)
LLaMA3.1-8B-Instruct	51.63	53.23	30.78	68.67	54.29	60.52	53.19	316.14
• MInference [21]	51.70	52.72	30.76	68.58	53.50	61.12	53.06	324.84
• FlexPrefill [22]	50.35	52.85	30.71	68.41	54.30	62.76	53.23	280.11
• XAttention [23]	49.96	51.98	31.22	68.07	48.50	55.75	50.91	133.85
• TCA-Attention (Ours)	52.28	52.83	30.84	68.40	54.86	61.82	53.51	120.96
Qwen2.5-7B-Instruct	48.75	52.24	27.81	65.00	52.00	61.14	51.16	268.55
• MInference [21]	48.80	52.37	27.64	64.67	51.50	62.08	51.18	292.33
• FlexPrefill [22]	49.08	52.16	27.86	65.18	52.00	62.20	51.41	244.37
• XAttention [23]	48.50	50.08	27.48	66.40	50.50	60.98	50.66	119.87
• TCA-Attention (Ours)	48.50	52.91	27.74	64.92	52.25	62.74	51.51	105.93

TABLE III: Comparisons on RULER [43] across efficient-attention methods under LLaMA3.1-8B and Qwen2.5-7B models. We report the attention computation latency at 128K contexts.

Method	4K	8K	16K	32K	64K	128K	Average	Latency (ms)
LLaMA3.1-8B-Instruct	96.74	94.03	92.02	84.17	81.32	76.89	87.52	1282.05
• MInference [21]	96.54	94.06	91.37	85.79	83.03	54.13	84.15	837.09
• FlexPrefill [22]	95.99	93.67	92.73	88.14	81.14	74.67	87.72	1021.47
• XAttention [23]	96.15	93.95	93.71	90.90	83.35	72.57	88.44	504.00
• TCA-Attention (Ours)	96.31	95.38	93.92	86.38	82.89	77.46	88.72	453.22
Qwen2.5-7B-Instruct	96.00	94.85	91.77	89.85	70.38	52.92	82.63	1101.01
• MInference [21]	96.08	94.92	91.69	89.92	70.46	51.62	82.45	771.61
• FlexPrefill [22]	95.62	94.31	92.00	88.23	70.23	52.15	82.09	892.55
• XAttention [23]	95.45	92.91	92.04	88.84	68.84	55.36	82.24	441.36
• TCA-Attention (Ours)	96.00	94.77	91.92	90.08	69.85	52.31	82.49	399.09

the retained attention mass of the selected tokens. This establishes a principled foundation for the core-context selection mechanism used in both prefiling and decoding stages.

Let Att_i denote the full attention output for the i -th query, and let $\widetilde{\text{Att}}_i$ denote the corresponding approximation computed using only the selected global and local tokens. The following theorem provides a formal upper bound.

Theorem 1 (Error Bound for a Single Query). *Let $\gamma_i = \sum_{j \notin S_{\text{total}}} \frac{\exp(\mathbf{A}_{ij})}{Z_i}$ be the total probability mass that the full softmax assigns to tokens omitted by TCA-Attention. Here, S_{total} denotes the complete set of token indices selected by our method, including the global subset and local subset. Then the approximation error satisfies:*

$$|\text{Att}_i - \widetilde{\text{Att}}_i|_1 \leq 2\gamma_i \cdot |\mathbf{V}|_\infty,$$

where $|\mathbf{V}|_\infty$ is the maximum absolute entry in value matrix.

Theorem 1 shows that the error scales linearly with γ_i , i.e., the fraction of probability mass on unselected tokens. Since the offline sparsity determination in TCA-Attention enforces that the aggregated retained attention mass exceeds a threshold τ , we effectively ensure $\gamma_i \leq 1 - \tau$. Thus, the bound is directly governed by a tunable hyperparameter, providing explicit controllability. Furthermore, the guaranteed inclusion of the most recent local tokens prevents large γ_i values in generative decoding, while head-specific sparsity budgets ensure that redundancy is eliminated only where error tolerance is naturally higher.

The full proof of Theorem 1, along with additional derivations and discussion, is provided in Appendix A. Together, these results establish that TCA-Attention is a principled approximation mechanism whose deviation from full attention is not only bounded, but also systematically regulated by its sparsity configuration process.

V. EXPERIMENTS

A. Experimental Setup

Models. We evaluate our method on two state-of-the-art large language models: **Qwen2.5-7B-Instruct** [56] and **LLaMA3.1-8B-Instruct** [6]. Both support context windows of up to 128K tokens and employ grouped-query attention (GQA). Qwen2.5-7B-Instruct demonstrates enhanced capabilities in multilingual understanding, mathematical reasoning, and code generation. LLaMA3.1-8B-Instruct is optimized for dialogue and generation tasks across multiple languages.

Compared Methods. We compare TCA-Attention with three training-free attention methods: **1) Minference** [21]: This method employs offline determination to select optimal sparse attention patterns per attention head, combined with online dynamic adjustment of computation regions for each pattern. We use the officially released implementation in all experiments. **2) FlexPrefill** [22]: This approach dynamically selects between Query-Aware and Vertical-Slash attention patterns per head, while adaptively determining the required Key-Value indices for computation. In our experiments, we use the official implementation and follow the original paper's settings: $\gamma=0.9$, $\tau=0.1$, with a minimum retained budget of 1,024 tokens and a block size of 128 across all evaluated models. **3) XAttention** [23]: This method employs an anti-diagonal scoring pattern to select sparse attention blocks, reducing computation by processing only the selected regions. In our experiments, we follow the original paper's optimal setting with stride $S = 8$. For LLaMA3.1-8B, we adopt the officially released set of minimum thresholds to determine block selection. For Qwen2.5-7B, we set the threshold to 0.9 to preserve more contextual information. This constitutes a fair comparison within the same category of techniques.

TABLE IV: Comparisons on short-context tasks.

Method	MMLU	GSM-8K	HumanEval
LLaMA3.1-8B-Instruct	69.38	83.85	68.29
• MInference [21]	69.14	84.08	67.30
• FlexPrefill [22]	69.16	84.15	67.07
• XAttention [23]	69.21	84.15	67.39
• TCA-Attention (Ours)	69.21	84.23	67.46
Qwen2.5-7B-Instruct	74.22	79.68	81.71
• MInference [21]	74.14	80.29	79.88
• FlexPrefill [22]	74.23	80.36	81.10
• XAttention [23]	74.20	79.30	80.49
• TCA-Attention (Ours)	74.26	80.44	81.10

TABLE V: Comparisons on OlympiadBench.

Method	Math	Physics	Average
LLaMA3.1-8B-Instruct	13.84	8.85	11.84
• MInference [21]	13.64	9.73	12.08
• FlexPrefill [22]	13.39	8.85	11.57
• XAttention [23]	13.71	9.30	11.95
• TCA-Attention (Ours)	13.19	10.59	12.15
Qwen2.5-7B-Instruct	38.85	19.73	31.20
• MInference [21]	38.79	19.28	30.99
• FlexPrefill [22]	38.73	19.06	30.86
• XAttention [23]	38.94	18.84	30.90
• TCA-Attention (Ours)	38.82	19.95	31.27

TABLE VI: Results on MT-Bench-101 with Qwen2.5-7B-Instruct.

Method	GR	IC	AR	FR	MR	CC	TS	CR	SA	SI	CM	PI	SC	Average
Qwen2.5-7B-Instruct	8.20	7.71	9.49	9.56	7.57	9.89	8.90	9.50	9.29	8.26	9.19	8.64	9.42	8.90
• FlexPrefill [22]	7.81	7.67	9.55	9.57	7.23	9.91	9.40	9.57	9.22	8.33	9.27	8.63	9.53	8.90
• TCA-Attention (Ours)	8.50	7.71	9.56	9.74	7.48	9.91	9.39	9.49	9.26	8.38	9.20	8.62	9.43	8.97

TABLE VII: Performance comparisons across prefilling and decoding on LongBench-E.

Method	Sin. Doc. QA	Mul. Doc. QA	Sum.	Few Shot	Syn.	Code	Average
MInference [21] + SnapKV [24]	49.66	53.16	29.90	67.60	51.72	58.25	51.72
FlexPrefill [22] + CAKE [25]	50.27	52.87	30.77	66.96	52.25	60.24	52.23
DuoAttention [29]	51.05	52.70	29.70	67.27	52.25	59.86	52.14
TCA-Attention (Ours)	50.71	53.11	30.24	67.93	52.59	60.05	52.44

Implementation Details. We integrate the proposed TCA-Attention with existing LLMs through a **plug-and-play** replacement of full self-attention, requiring **no architectural modifications** or **parameter updates**. All experiments are conducted on a server with $8 \times$ NVIDIA A800 GPUs (80 GB VRAM each). All reported latency measurements are obtained using a single A800 GPU. The vanilla attention baseline is implemented using the highly optimized FlashAttention-2 kernel [34]. To align with established practices for long-context evaluation [21]–[23], latency is measured at batch size = 1 with input sequences up to 128K tokens. The offline sparsity threshold is set to $\tau = 0.9$. Block size b and window size w are set to 128 and 4096 for all models, respectively. For sparsity configuration determination, we adopt 14 candidate configurations for all LLMs. These candidate configurations are generated using the log-Gaussian sampling (Eq. 3) with μ sampled from $\log_2(1)$ to $\log_2(128)$. The optimal configuration per head is determined via the offline procedure in Section IV-A. All hyperparameters are held constant across experiments to ensure reproducibility.

Datasets. We conduct a comprehensive evaluation across long-context modeling, short-context understanding, complex reasoning, and multi-turn dialogue tasks. For long-context assessment, we employ **LongBench-E** [42], which includes 14 diverse tasks for long-context understanding, and **RULER** [43], a synthetic benchmark built on the Needle-in-a-Haystack paradigm. For short-context evaluation, we use **MMLU** [57] (multi-subject knowledge), **GSM8K** [58] (mathematical reasoning), and **HumanEval** [59] (code generation). Additionally, we evaluate on **OlympiadBench** [60] for advanced scientific reasoning using Olympiad-level problems. We also evaluate on **MT-Bench-101** [61] to validate the effectiveness of TCA-Attention in multi-turn conversations. Refer to Appendix C for more details.

B. Comparisons with State-of-the-Art Methods

Comparisons on LongBench-E. In Table III, TCA-Attention achieves the highest average score among sparse attention methods. For LLaMA3.1-8B, it attains a $2.6 \times$ speedup (120.96 ms vs. 316.14 ms) at 64K context while outperforming XAttention by 2.6 in average score. On Qwen2.5-7B, it delivers the best performance with the lowest latency ($2.5 \times$ speedup). These results validate our method’s effectiveness and architectural adaptability without compromising accuracy.

Comparisons on RULER. In Table III, TCA-Attention achieves superior efficiency and accuracy in long context understanding across different context lengths. On LLaMA3.1-8B-Instruct and Qwen2.5-7B-Instruct, TCA-Attention attains the highest average score with up to $2.8 \times$ speedup compared to vanilla self-attention (0.45s vs 1.28s), outperforming the strongest counterpart method XAttention despite faster computation. These results confirm that our dynamic sparsity mechanism effectively maintains performance while accelerating computation across varying sequence lengths.

Comparisons on Short-context Tasks. In Table IV, our TCA-Attention achieves competitive results on LLaMA3.1-8B-Instruct-128K and outperforms all baselines on Qwen2.5-7B-Instruct-128K, demonstrating robust cross-model generalization. Notably, it preserves the capability of the original model, validating that our method effectively retains critical information without fine-tuning.

Comparisons on Reasoning Tasks. To evaluate the effectiveness of our TCA-Attention on challenging reasoning tasks requiring long-context understanding, we conduct experiments on the OlympiadBench benchmark, which includes advanced problems in Mathematics and Physics. As shown in Table V, our method demonstrates competitive and often superior performance compared to existing sparse attention methods. On both LLaMA3.1-8B-Instruct and Qwen2.5-7B-Instruct, TCA-

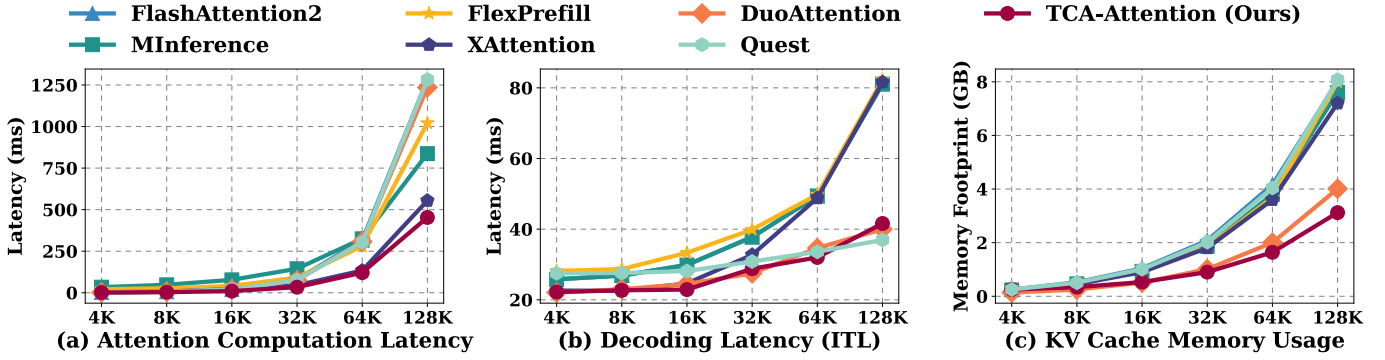


Fig. 3: Comparisons in terms of computational and storage overhead on LLaMA3.1-8B-Instruct. Attention computation latency is the time to compute a single attention layer. “ITL” (inter-token latency) is the time between generating consecutive tokens (except for the first token) in decoding.

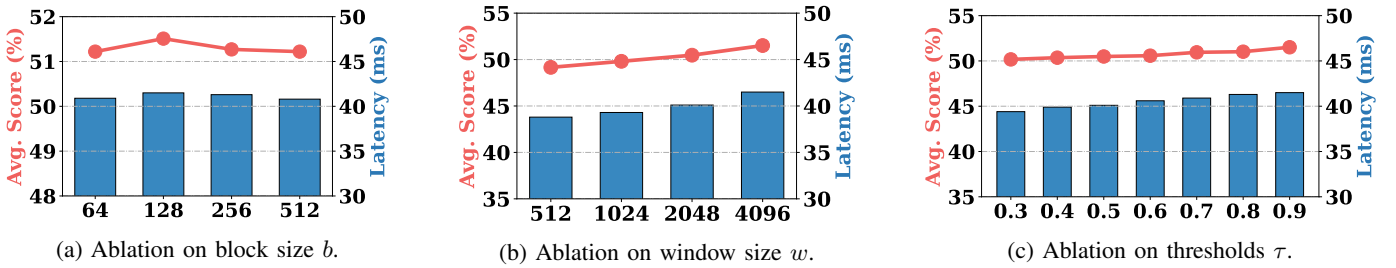


Fig. 4: Ablations on hyperparameters. We report the average score on LongBench-E and the decoding latency.

Attention achieves the highest average score, outperforming all compared methods. Notably, TCA-Attention obtains the highest score in Physics, while maintaining competitive performance in Mathematics. This result further confirms the strong capability of TCA-Attention in handling complex reasoning.

Comparisons on Multi-turn Conversation. To evaluate the model’s capability in maintaining contextual coherence and reasoning across multi-turn interactions, we conduct experiments on MT-Bench-101 [61]. As shown in Table VI, TCA-Attention achieves an average score of 8.97, surpassing both the full-attention baseline (8.90) and strong sparse methods such as FlexPrefill (8.90). This advantage is consistent across key conversational dimensions, including generation, reasoning, and safety. The results demonstrate that our TCA-Attention effectively preserves essential dialogue context across turns, validating its robustness and practicality for real-world conversational applications.

Comparisons on Computational Efficiency. As shown in Figure 3, TCA-Attention achieves a $2.8\times$ prefilling speedup and a $2.1\times$ decoding speedup over vanilla self-attention at 128K context length on LLaMA3.1-8B-Instruct. By dynamically selecting tokens, it also reduces KV cache memory by 61% (3.12 GB vs. 8.00 GB). MInference, FlexPrefill, and XAttention primarily target prefilling acceleration; consequently, they provide limited or no improvement in decoding latency and do not reduce KV cache memory usage. Quest and DuoAttention achieve slightly lower decoding ITL than TCA-Attention (36.94 ms and 40.00 ms vs. 41.63 ms), but their prefilling attention latency remains close to that of full attention. This is because Quest does not target prefilling

acceleration, while the official DuoAttention implementation directly uses FlashAttention-2 for dense prefilling computation. Moreover, Quest retains the full 8.00 GB KV cache, compared with 3.12 GB for TCA-Attention. Overall, these results show that TCA-Attention provides a more balanced efficiency improvement across prefilling, decoding, and KV cache memory.

Performance Comparisons Across Prefilling and Decoding. To validate the effectiveness of our unified framework across *both prefilling and decoding stages*, we further conduct experiments on LongBench-E by combining the strongest prefilling-stage baselines with KV cache compression methods. The results in Table VII demonstrate that our approach achieves superior performance across most tasks. Although DuoAttention aims to reduce latency in both prefilling and decoding stages, the official DuoAttention implementation employs a standard FlashAttention-2 kernel for prefilling, thus failing to reduce computation in this computationally intensive stage. This highlights that our method provides a more efficient acceleration method for long-context processing.

Performance Comparisons with KV Cache Retrieval Methods. We further compare TCA-Attention with Quest [26], a representative query-aware KV cache retrieval method. We use its official implementation with a KV budget of 4,096 and evaluate both methods on LongBench-E using LLaMA3.1-8B-Instruct. As shown in Table VIII, TCA-Attention achieves a slightly higher average score than Quest (52.44 vs. 52.17). Quest reduces the decoding ITL to 36.94 ms by retrieving only the relevant KV pages for each query, but it does not accelerate prefilling or reduce KV cache memory usage. In comparison,

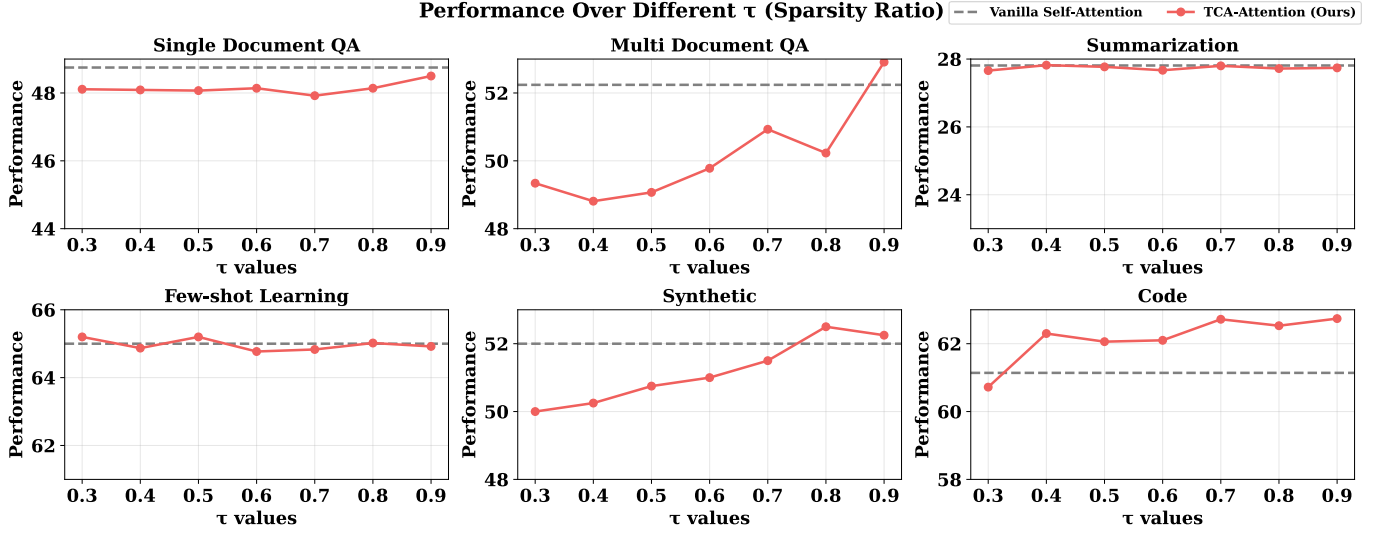


Fig. 5: Performance comparisons across different τ values (corresponding to different sparsity ratios) on LongBench-E tasks.

TABLE VIII: Performance and efficiency comparisons on LongBench-E using LLaMA3.1-8B-Instruct. The efficiency metrics are measured at a 128K context length. “ITL” denotes the inter-token latency during decoding.

Method	Average Score $\uparrow$	Prefilling Latency (ms) $\downarrow$	ITL (ms) $\downarrow$	KV Cache (GB) $\downarrow$
LLaMA3.1-8B-Instruct	53.19	1282.05	85.35	8.00
• Quest [26]	52.17	1256.41 (1.0 $\times$)	36.94 (2.3$\times$)	8.00 (0.0% $\downarrow$)
• TCA-Attention (Ours)	52.44	453.22 (2.8$\times$)	41.63 (2.1 $\times$)	3.12 (61%$\downarrow$)

TABLE IX: Performance with different α values.

α	0.1	0.3	0.5	0.7	0.9
Avg.	51.14	51.25	51.51	51.24	51.13

TABLE X: Performance with different calibration datasets.

Datasets	Domain	Average Score
SlimPajama	General web text	51.54 $\pm$ 0.03
GovReport	Governmental docs	51.56 $\pm$ 0.04
McEval	Programming code	51.55 $\pm$ 0.02

TABLE XI: Ablation on the number of query tokens used for importance estimation. We report the average LongBench-E score and the TCA-Attention computation latency at a 128K context length, measured on a single NVIDIA A800 GPU.

#Queries k	1	2	4	8	16	32
Average Score	51.51	51.55	51.59	51.69	51.62	51.65
Latency (ms)	399.09	400.92	405.21	414.99	425.72	439.68

TCA-Attention reduces the prefilling latency from 1282.05 ms to 453.22 ms, reduces the decoding ITL to 41.63 ms, and decreases KV cache memory usage from 8.00 GB to 3.12 GB. These results show that KV cache retrieval is effective for decoding-time memory access reduction, while TCA-Attention provides efficiency improvements across prefilling, decoding, and KV cache storage.

C. Ablation Studies

We perform ablation studies on Qwen2.5-7B-Instruct-128K and report the average score on LongBench-E. Additional ablation results are provided in Appendix D.

Ablations on Block Size b . As shown in Figure 4a, the model achieves the highest average score of 51.51 with a block size of 128. Smaller block sizes may fail to capture sufficient contextual redundancy, while larger sizes risk over-compressing critical information.

Ablations on Local Window Size w . As shown in Figure 4b, the performance of the model improves consistently as the window size increases, achieving the highest average score

of 51.51 with a window size of 4096. Expanding the window size yields consistent performance gains by preserving richer local context details, with latency increasing only marginally from 38.8 ms to 41.5 ms.

Ablations on Threshold τ . We analyze the impact of the threshold τ on model performance, where lower τ values correspond to higher compression rates. As shown in Figure 4c, performance generally improves as τ increases from 0.3 to 0.9, with peak accuracy attained at $\tau = 0.9$. This reflects that retaining more contextual information benefits overall effectiveness, while latency increases only marginally. Detailed per-task results (Figure 5) further reveal that tasks relying on localized context, such as Single-Document QA and Summarization, remain robust even under high compression ($\tau = 0.3$, 84.69% compression), suffering only a slight average drop. In contrast, tasks requiring global reasoning across the sequence, such as Multi-Document QA and Code, are more sensitive to aggressive compression. These observations suggest a practical guideline: conservative thresholds ($\tau = 0.6$ or 0.9) are advisable for globally-dependent tasks, whereas locally-dependent tasks can tolerate stronger compression to maximize efficiency with minimal performance loss.

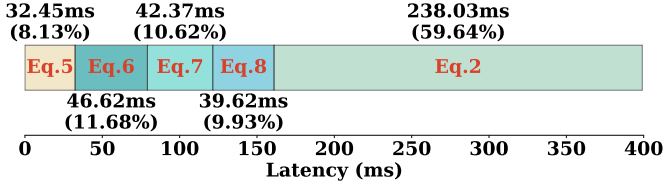


Fig. 6: Latency breakdown of TCA-Attention components at 128K context length with a total of 399.09ms.

Ablation Study on Balancing Parameter α . We analyze the sensitivity of TCA-Attention to the balance parameter α in our redundancy metric. Results in Table IX show consistent performance across a wide α range (0.1–0.9), demonstrating strong robustness. The optimal average score (51.51 on LongBench-E) occurs at $\alpha = 0.5$, while even extreme values ($\alpha = 0.1$ or $\alpha = 0.9$) cause minimal degradation (≤ 0.38). This insensitivity to precise tuning confirms the practical deployability of our method.

Ablation Study on Calibration Dataset. We conduct an ablation study to evaluate TCA-Attention’s sensitivity to calibration dataset choice. As shown in Table X, our method achieves consistent performance across different calibration datasets and sizes. When calibrated on diverse domains, including general web text (SlimPajama), long-form governmental documents (GovReport), and programming code (McEval), the average score on LongBench-E remains remarkably stable (51.54–51.56). More importantly, this stability persists even with minimal calibration: using just a single input sequence (size=1), our method achieves virtually identical performance when randomly sampling five samples, with variations of ≤ 0.04 . These results demonstrate two critical properties: 1) head-specific sparsity transfers well across domains, and 2) the determined configurations are remarkably stable across different inputs within the same model, requiring only minimal calibration data. This makes TCA-Attention highly practical for real-world deployment, as it eliminates the need for domain-specific or extensive calibration datasets while maintaining promising performance.

Ablation on the Number of Query Tokens. We generalize Eq. 5 by averaging the importance scores computed from the last k query tokens as $\mathbf{s}^{(k)} = \frac{1}{k} \sum_{r=L-k+1}^L \text{softmax} \left(\frac{\mathbf{Q}_{r,:} \mathbf{K}^\top}{\sqrt{d_h}} \right)$. We evaluate $k \in \{1, 2, 4, 8, 16, 32\}$ on LongBench-E using Qwen2.5-7B-Instruct. As shown in Table XI, the average score varies only slightly across different values of k : the best result at $k = 8$ exceeds that at $k = 1$ by only 0.18 points (51.69 vs. 51.51). In contrast, the attention computation latency increases from 399.09 ms at $k = 1$ to 414.99 ms at $k = 8$ and 439.68 ms at $k = 32$. We adopt $k = 1$ as the default setting, as it provides performance close to the best multi-query variant with the lowest latency.

D. Latency Breakdown Analysis

To validate the practical efficiency of TCA-Attention, we provide a detailed latency breakdown of our Triton implementation at 128K context length, as shown in Figure 6. The importance score calculation (Eq. 5, 8.13%) and other

auxiliary operations, including redundancy metric computation (Eq. 6), sorting (Eq. 7), and token selection (Eq. 8), collectively contribute less than 40% of total latency. The latency is dominated by attention computation over fused global-local subsets (Eq. 2). Crucially, this overhead is significantly outweighed by the computational savings achieved through sparsification, ultimately enabling a speedup of $2.8\times$ compared to full attention. The efficiency primarily stems from our block-level design, which minimizes the cost of dynamic token selection and redundancy estimation, as opposed to more expensive token-level processing.

VI. CONCLUSION

In this paper, we propose Training-free Context-adaptive Attention (TCA-Attention), a sparse attention mechanism that enables efficient long-context modeling without any training or architectural changes. It employs two lightweight phases, *i.e.*, an offline head-specific sparsity determination phase and an online core context selection phase, to achieve input-adaptive sparsity. Theoretical analysis guarantees bounded approximation error, and extensive experiments validate that TCA-Attention achieves significant speedup (up to $2.8\times$) and KV cache reduction (61%) at 128K context while maintaining competitive performance across diverse benchmarks. The plug-and-play nature of TCA-Attention makes it a practical solution for LLMs in long-context scenarios without retraining or architectural changes.

ACKNOWLEDGMENT

This work was supported in part by the Joint Funds of the National Natural Science Foundation of China under Grant No.U24A20327, the Guangdong S&T Program under Grant No.2026B0101110001, the Guangdong Basic and Applied Basic Research Foundation under Grant No.2026A151010388, the Postdoctoral Fellowship Program of CPSF under Grant GZC20251043, and the project of 62536003, PCL2025A14.

REFERENCES

- [1] E. De Santis, A. Martino, and A. Rizzi, “Human versus machine intelligence: Assessing natural language generation models through complex systems theory,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 7, pp. 4812–4829, 2024.
- [2] B. Lin, Y. Nie, Z. Wei, Y. Zhu, H. Xu, S. Ma, J. Liu, and X. Liang, “Correctable landmark discovery via large models for vision-language navigation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 8534–8548, 2024.
- [3] J. Zhang, J. Huang, S. Jin, and S. Lu, “Vision-language models for vision tasks: A survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 8, pp. 5625–5644, 2024.
- [4] R. Zeng, W. Huang, M. Tan, Y. Rong, P. Zhao, J. Huang, and C. Gan, “Graph convolutional networks for temporal action localization,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 7094–7103.
- [5] H. T. et al., “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [6] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [7] OpenAI, “GPT-4 technical report,” *arXiv preprint*, 2023.
- [8] A. Jaech, A. Kalai, A. Lerer, A. Richardson, A. El-Kishky, A. Low, A. Helyar, A. Madry, A. Beutel, A. Carney *et al.*, “Openai o1 system card,” *arXiv preprint arXiv:2412.16720*, 2024.

- [9] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning,” *arXiv preprint*, 2025.
- [10] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, 2022, pp. 24 824–24 837.
- [11] M. Besta, F. Memedi, Z. Zhang, R. Gerstenberger, G. Piao, N. Blach, P. Nyczyk, M. Copik, G. Kwasniewski, J. Müller, L. Gianinazzi, A. Kubicek, H. Niewiadomski, A. O’Mahony, O. Mutlu, and T. Hoefler, “Demystifying chains, trees, and graphs of thoughts,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 47, no. 12, pp. 10 967–10 989, 2025.
- [12] N. Xu, D. Xue, S. Qian, Q. Fang, and J. Hu, “Prompting large language models for automatic question tagging,” *Mach. Intell. Res.*, vol. 22, no. 5, pp. 917–928, 2025.
- [13] R. Pasunuru, A. Celikyilmaz, M. Galley, C. Xiong, Y. Zhang, M. Bansal, and J. Gao, “Data augmentation for abstractive query-focused multi-document summarization,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 15, 2021, pp. 13 666–13 674.
- [14] L. Zhang, Y. Zhong, Q. Zheng, J. Liu, Q. Wang, J. Wang, and X. Chang, “TDGI: translation-guided double-graph inference for document-level relation extraction,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 47, no. 4, pp. 2647–2659, 2025.
- [15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.
- [16] Y. Chen, Z. You, S. Zhang, H. Li, Y. Li, Y. Wang, and M. Tan, “Core context aware transformers for long context language modeling,” in *International Conference on Machine Learning*, 2025.
- [17] S. Zhang, Z. You, Y. Chen, Z. Wen, Q. Wang, Z. Qiu, Y. Li, and M. Tan, “Curse of high dimensionality issue in transformer for long context modeling,” in *International Conference on Machine Learning*, 2025.
- [18] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang *et al.*, “Big bird: Transformers for longer sequences,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 17 283–17 297, 2020.
- [19] I. Beltagy, M. E. Peters, and A. Cohan, “Longformer: The long-document transformer,” *arXiv preprint arXiv:2004.05150*, 2020.
- [20] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” in *International Conference on Learning Representations*, 2024.
- [21] H. Jiang, Y. Li, C. Zhang, Q. Wu, X. Luo, S. Ahn, Z. Han, A. H. Abdi, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, “MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention,” in *Advances in Neural Information Processing Systems*, 2024.
- [22] X. Lai, J. Lu, Y. Luo, Y. Ma, and X. Zhou, “Flexprefill: A context-aware sparse attention mechanism for efficient long-sequence inference,” in *International Conference on Learning Representations*, 2025.
- [23] R. Xu, G. Xiao, H. Huang, J. Guo, and S. Han, “Xattention: Block sparse attention with antidiagonal scoring,” in *International Conference on Machine Learning*, 2025.
- [24] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, “Snapkv: Llm knows what you are looking for before generation,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 22 947–22 970, 2024.
- [25] Z. Qin, Y. Cao, M. Lin, W. Hu, S. Fan, K. Cheng, W. Lin, and J. Li, “Cake: Cascading and adaptive kv cache eviction with layer preferences,” in *International Conference on Learning Representations*, 2025.
- [26] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, “QUEST: query-aware sparsity for efficient long-context LLM inference,” in *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21–27, 2024*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 47 901–47 911.
- [27] H. Zhang, X. Ji, Y. Chen, F. Fu, X. Miao, X. Nie, W. Chen, and B. Cui, “Pqcache: Product quantization-based kvcache for long context LLM inference,” *Proc. ACM Manag. Data*, vol. 3, no. 3, pp. 201:1–201:30, 2025.
- [28] Z. Chen, R. Sadhukhan, Z. Ye, Y. Zhou, J. Zhang, N. Nolte, Y. Tian, M. Douze, L. Bottou, Z. Jia, and B. Chen, “Magicpig: LSH sampling for efficient LLM generation,” in *International Conference on Learning Representations*, 2025.
- [29] G. Xiao, J. Tang, J. Zuo, S. Yang, H. Tang, Y. Fu, S. Han *et al.*, “Duoat-tention: Efficient long-context llm inference with retrieval and streaming heads,” in *International Conference on Learning Representations*, 2025.
- [30] S. Yang, J. Guo, H. Tang, Q. Hu, G. Xiao, J. Tang, Y. Lin, Z. Liu, Y. Lu, and S. Han, “Lserve: Efficient long-sequence llm serving with unified sparse attention,” in *Eighth Conference on Machine Learning and Systems*, 2025.
- [31] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, “Transformers are rnns: Fast autoregressive transformers with linear attention,” in *International Conference on Machine Learning*. PMLR, 2020, pp. 5156–5165.
- [32] Y. Sun, L. Dong, S. Huang, S. Ma, Y. Xia, J. Xue, J. Wang, and F. Wei, “Retentive network: A successor to transformer for large language models,” *arXiv preprint arXiv:2307.08621*, 2023.
- [33] Z. Zhuang, M. Tan, B. Zhuang, J. Liu, Y. Guo, Q. Wu, J. Huang, and J. Zhu, “Discrimination-aware channel pruning for deep neural networks,” *Advances in neural information processing systems*, vol. 31, 2018.
- [34] T. Dao, “Flashattention-2: Faster attention with better parallelism and work partitioning,” in *International Conference on Learning Representations*, 2024.
- [35] K. Hong, G. Dai, J. Xu, Q. Mao, X. Li, J. Liu, K. Chen, Y. Dong, and Y. Wang, “Flashdecoding++: Faster large language model inference with asynchronization, flat gemm optimization, and heuristics,” *Proceedings of Machine Learning and Systems*, vol. 6, pp. 148–161, 2024.
- [36] Z. Wang, T. Luo, C. Liu, W. Liu, R. S. M. Goh, and W. Wong, “Enabling energy-efficient deployment of large language models on memristor crossbar: A synergy of large and small,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 47, no. 2, pp. 916–933, 2025.
- [37] Y. Liu, Y. Wu, G. Sun, L. Zhang, A. Chhatkuli, and L. V. Gool, “Vision transformers with hierarchical attention,” *Mach. Intell. Res.*, vol. 21, no. 4, pp. 670–683, 2024.
- [38] Z. Wan, X. Wu, Y. Zhang, Y. Xin, C. Tao, Z. Zhu, X. Wang, S. Luo, J. Xiong, and M. Zhang, “D2o: Dynamic discriminative operations for efficient generative inference of large language models,” in *International Conference on Learning Representations*, 2025.
- [39] J. Yuan and H. e. Gao, “Native sparse attention: Hardware-aligned and natively trainable sparse attention,” in *Association for Computational Linguistics*. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 23 078–23 097.
- [40] E. Lu, Z. Jiang, J. Liu, Y. Du, T. Jiang, C. Hong, S. Liu, W. He, E. Yuan, Y. Wang *et al.*, “Moba: Mixture of block attention for long-context llms,” *arXiv preprint arXiv:2502.13189*, 2025.
- [41] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.
- [42] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, “Longbench: A bilingual, multitask benchmark for long context understanding,” *arXiv preprint arXiv:2308.14508*, 2023.
- [43] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg, “Ruler: What’s the real context size of your long-context language models?” *arXiv preprint arXiv:2404.06654*, 2024.
- [44] X. Liu, K. Lv, Q. Guo, H. Yan, C. He, X. Qiu, and D. Lin, “Longwan-juan: Towards systematic measurement for long text quality,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 5709–5725.
- [45] L. Chen, Z. Liu, W. He, Y. Zheng, H. Sun, Y. Li, R. Luo, and M. Yang, “Long context is not long at all: A prospector of long-dependency data for large language models,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 8222–8234.
- [46] Y. Fu, R. Panda, X. Niu, X. Yue, H. Hajishirzi, Y. Kim, and H. Peng, “Data engineering for scaling language models to 128k context,” *arXiv preprint arXiv:2402.10171*, 2024.
- [47] S. An, Z. Ma, Z. Lin, N. Zheng, J.-G. Lou, and W. Chen, “Make your llm fully utilize the context,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 62 160–62 188, 2024.
- [48] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, “Roformer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, p. 127063, 2024.
- [49] B. Peng and J. Quesnelle, “Ntk-aware scaled rope allows llama models to have extended (8k+) context size without any fine-tuning and minimal perplexity degradation,” 2023.
- [50] B. Peng, J. Quesnelle, H. Fan, and E. Shippole, “YaRN: Efficient context window extension of large language models,” in *International Conference on Learning Representations*, 2024.
- [51] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” in *First conference on language modeling*.

- [52] Y. Sun, X. Li, K. Dalal, J. Xu, A. Vikram, G. Zhang, Y. Dubois, X. Chen, X. Wang, S. Koyejo *et al.*, “Learning to (learn at test time): Rnns with expressive hidden states,” in *Forty-second International Conference on Machine Learning*.
- [53] P. Manakul and M. J. F. Gales, “Long-span summarization via local attention and content selection,” in *Association for Computational Linguistics*, C. Zong, F. Xia, W. Li, and R. Navigli, Eds., 2021, pp. 6026–6041.
- [54] B. Yang, L. Wang, D. F. Wong, S. Shi, and Z. Tu, “Context-aware self-attention networks for natural language processing,” *Neurocomputing*, vol. 458, pp. 157–169, 2021.
- [55] S. A. Rhoades, “The herfindahl-hirschman index,” *Fed. Res. Bull.*, vol. 79, p. 188, 1993.
- [56] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei *et al.*, “Qwen2. 5 technical report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [57] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Measuring massive multitask language understanding,” in *International Conference on Learning Representations*, 2021.
- [58] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [59] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [60] C. He, R. Luo, Y. Bai, S. Hu, Z. Thai, J. Shen, J. Hu, X. Han, Y. Huang, Y. Zhang *et al.*, “Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems,” in *Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 3828–3850.
- [61] G. Bai, J. Liu, X. Bu, Y. He, J. Liu, Z. Zhou, Z. Lin, W. Su, T. Ge, B. Zheng *et al.*, “Mt-bench-101: A fine-grained benchmark for evaluating large language models in multi-turn dialogues,” in *Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 7421–7454.



Zhijie Qiu received the B.E. degree in Software Engineering from the School of Software Engineering, South China University of Technology, China, in 2024. He is currently working toward the M.S. degree in the School of Software Engineering, South China University of Technology, China. His research interests include deep learning and model efficient inference.



Tingyu Wu is currently pursuing the B.E. degree in the School of Software Engineering, South China University of Technology, China. His research interests include large language model architectures and LLM-based agents.



Yingjian Li received the B.Sc., M.Sc., and Ph.D. degrees from Harbin Institute of Technology, China, in 2016, 2018, and 2023, respectively. He was a visiting researcher of the Nanyang Technological University (NTU). Now he is an Assistant Research Fellow at Institute of Perceptual Intelligence, Pengcheng Laboratory, Shenzhen, China. He has published more than 20 journal (IEEE TIP, TMM, TAFFC, TCSVT, etc.) and conference (ACM-MM) papers. His research interests include computer vision, multimodal learning, and affective computing.



Zeng You received the B.E. degree in Software Engineering from the School of Software Engineering, South China University of Technology, China, in 2020. He is currently working toward the Ph.D. degree in the School of Future Technology, South China University of Technology, China. His research interests include deep learning, video understanding, and efficient attention. He has published papers in top venues, including ICML, AAAI, IEEE TIP, and IEEE TCSVT. He has been invited as a reviewer for NeurIPS, AAAI, TII, and TCSVT.



Yaowei Wang (Member, IEEE) is a professor at the Harbin Institute of Technology, Shenzhen (HITSZ) and the Peng Cheng Laboratory. His research interests include artificial intelligence and large-scale video intelligent perception. He has published over 100 papers in top-tier international journals and conferences and holds more than 50 authorized patents. He is a recipient of the “National High-Level Talent” honor. His work has been recognized with major awards, including the National Technological Invention Award (Second Prize), and both the Science and Technology Progress Award (First Prize) and the Technology Invention Award (First Prize) from the Chinese Institute of Electronics. He serves as an Associate Editor for IEEE TCSVT and the chair of the IEEE Digital Retina System Working Group.



CVPR, ICCV, ECCV and AAAI.

Yaofu Chen is Post-doctoral Researcher with School of Future Technology at South China University of Technology. He received his Ph.D. degree in the School of Software Engineering in 2024 from South China University of Technology in Guangzhou, China. His research interests include neural architecture search and test-time adaptation. He has published papers in top venues, including ICML, ICLR, CVPR, AAAI, IEEE TCSVT and Neural Networks. He has been invited as a reviewer for top-tier conferences including ICLR, ICML, NeurIPS,



Shuhai Zhang is currently a Ph.D. candidate at South China University of Technology, China. His research interests are broadly in machine learning and mainly focus on large language models, model compression, and adversarial robustness. He has published papers in IEEE TIP, Neural Networks, TCSVT, NeurIPS, ICCV, ICML, ICLR, CVPR.



He also serves as an Associate Editor for IEEE Transactions on Pattern Analysis and Machine Intelligence. His research interests include machine learning, sparse analysis, deep learning, and large-scale optimization.